

frame_table[]

kpage | upage | owner

frame_alloc(flags, upage)

frame_free(kpage)

Relatorio de Deciso es de Projeto

Projeto 3 — Frame Table (Memoria Virtual)

Centro de Informatica — Universidade Federal de Pernambuco

Disciplina: Sistemas Operacionais

2025 / 2026

1. Sumario Executivo

Este relatório documenta as decisões de projeto da implementação da **Frame Table** — primeira peça do subsistema de memória virtual do PintOS (Projeto 3). Antes desta implementação, nenhum código de memória virtual existia no projeto e o **Makefile.build** nem compilava a pasta **vm/**. Os dois arquivos criados, **vm/frame.h** e **vm/frame.c**, estabelecem a estrutura de dados global que rastreia todos os frames físicos em uso por processos de usuário, fornecendo a base sobre a qual evicção, swap e a tabela de páginas suplementar serão construídos.

Arquivo	Status	Função
vm/frame.h	Novo	Define struct frame_entry e protótipos públicos.
vm/frame.c	Novo	Implementa frame_init, frame_alloc e frame_free.
Makefile.build	Modificado	Ativa a compilação de vm/frame.c no kernel.

2. Contexto — Por que uma Frame Table?

No PintOS sem memória virtual, cada chamada a **palloc_get_page(PAL_USER)** aloca um frame físico diretamente. O kernel não mantém nenhum registro de quais frames estão em uso, quem os possui ou a qual página virtual do usuário cada um está mapeado. Isso é suficiente enquanto a memória física não se esgota — mas o Projeto 3 exige:

- **Evicção:** quando **palloc** retorna NULL (sem frames livres), o kernel precisa escolher uma vítima, salvá-la no swap e reutilizar o frame. Para isso, precisa saber quais frames existem e quem os possui.
- **Tabela de páginas suplementar:** ao resolver um page fault, o kernel precisa alocar um frame e registrar o mapeamento `upage -> kpage`.
- **Atributos por frame:** o algoritmo de relógio (clock) precisa de bits de acesso por frame; o campo `owner` permite invalidar a entrada na page table do processo dono durante a evicção.

Invariante central: todo frame físico alocado para um processo de usuário deve ter uma entrada correspondente em `frame_table`. A frame table é a fonte de verdade sobre o estado da memória física do sistema.

3. vm/frame.h — Estrutura de Dados

O header define a única estrutura de dados do módulo e os três protótipos públicos que compõem a interface da frame table.

vm/frame.h — struct frame_entry

```
struct frame_entry {
    void *kpage; /* Endereço virtual do kernel deste frame. */
    void *upage; /* Endereço virtual de usuário mapeado. */
    struct thread *owner; /* Thread dona deste frame. */
    struct list_elem elem; /* No na lista global frame_table. */
};

void frame_init (void);
void *frame_alloc (enum palloc_flags flags, void *upage);
```

```
void frame_free (void *kpage);
```

Campo	Tipo	Finalidade
kpage	void *	Endereço virtual do kernel retornado por <code>palloc_get_page</code> . Usado como chave de busca em <code>frame_free</code> .
upage	void *	Endereço virtual do usuário que será mapeado para este frame via <code>pagedir_set_page</code> .
owner	struct thread *	Thread que possui o frame. Necessário para evicão futura: permite invalidar a PTE do processo dono.
elem	struct list_elem	Nó de lista encadeada. Permite inserir e remover <code>frame_entry</code> da lista global <code>frame_table</code> .

4. vm/frame.c — Implementação

4.1 Estado Global

vm/frame.c — variáveis estáticas

```
static struct list frame_table;
static struct lock frame_lock;
```

frame_table é a lista encadeada que contém uma **frame_entry** para cada frame de usuário em uso no sistema. **frame_lock** é o mutex que serializa toda inserção e remoção nessa lista, garantindo que operações concorrentes de threads diferentes não corrompam a estrutura.

Por que lista e não hash table? Para a fase atual — sem evicão — buscas na `frame_table` só ocorrem em `frame_free`, que é chamado relativamente raramente. Uma lista encadeada é suficiente e mais simples. Quando o algoritmo de relógio for implementado, uma hash table indexada por `kpage` poderá ser introduzida para $O(1)$ de busca.

4.2 frame_init()

vm/frame.c — `frame_init()`

```
void frame_init(void) {
    list_init(&frame_table);
    lock_init(&frame_lock);
}
```

Inicializa a lista e o lock. Deve ser chamada exatamente uma vez durante o boot do kernel, em **threads/init.c**, após **palloc_init()** e antes de qualquer processo de usuário ser criado. Uma struct list zerada não é um estado válido — **list_init** precisa ser chamado explicitamente, assim como ocorre com **list_init(&t->children)** no Projeto 2.

4.3 frame_alloc()

Função central do módulo. Aloca um frame físico do pool de usuário, cria e preenche uma **frame_entry**, e a insere na tabela global. Retorna o endereço virtual do kernel para que o chamador possa mapeá-lo ao espaço do usuário via **pagedir_set_page**.

vm/frame.c — frame_alloc()

```

void *frame_alloc(enum pallocc_flags flags, void *upage) {
    void *kpage = pallocc_get_page(PAL_USER | flags);
    if (kpage == NULL)
        return NULL;

    struct frame_entry *f = malloc(sizeof *f);
    if (f == NULL) {
        pallocc_free_page(kpage);
        return NULL;
    }
    f->kpage = kpage;
    f->upage = upage;
    f->owner = thread_current();
    lock_acquire(&frame_lock);
    list_push_back(&frame_table, &f->elem);
    lock_release(&frame_lock);
    return kpage;
}

```

Passo	Decisão	Motivo
1 — pallocc_get_page(PAL_USER flags)	PAL_USER somado automaticamente	Todo frame na tabela é de usuário por definição. Evita erro do chamador de omitir a flag.
2 — malloc(frame_entry)	Heap, não stack	A entrada precisa sobreviver ao retorno da função. Stack seria inválida após o retorno.
3 — pallocc_free_page se malloc falhar	Cleanup imediato	Sem a entrada na tabela, o frame não poderia ser rastreado nem liberado depois.
4 — owner = thread_current()	Registrado no momento da alocação	Simplifica a interface; o alocador é sempre a thread corrente.
5 — lock antes de list_push_back	Lock apenas na inserção	A página física já pertence ao chamador; só a lista precisa de proteção.

4.4 frame_free()

Recebe o endereço virtual do kernel (**kpage**), localiza a entrada correspondente na tabela, remove-a, libera a memória da struct e devolve o frame ao pallocc.

vm/frame.c — frame_free()

```

void frame_free(void *kpage) {
    lock_acquire(&frame_lock);
    struct frame_entry *target = NULL;
    for (struct list_elem *e = list_begin(&frame_table);
         e != list_end(&frame_table);
         e = list_next(e))
    {
        struct frame_entry *f = list_entry(e, struct frame_entry, elem);
        if (f->kpage == kpage) {
            target = f;
            list_remove(e);
            break;
        }
    }
}

```

```
    }  
  }  
  lock_release(&frame_lock);  
  if (target != NULL)  
    free(target);  
  palloc_free_page(kpage);  
}
```

Tres decisoes de projeto se destacam nesta funcao:

- **Lock liberado antes de free e palloc_free_page.** O lock protege apenas a integridade da lista — nao a pagina fisica em si. Liberar o lock antes de chamar free e palloc_free_page reduz o tempo de contencao, permitindo que outras threads acessem a tabela enquanto a limpeza final ocorre.
- **palloc_free_page sempre chamado, mesmo sem entrada na tabela.** pagedir_destroy pode liberar frames diretamente sem passar por frame_free. Chamar palloc_free_page incondicionalmente garante que o frame fisico seja sempre devolvido ao pool — o invariante e que palloc_free_page seja chamado exatamente uma vez por frame alocado.
- **Guarda contra double-free na struct.** Se kpage nao estiver na tabela, target permanece NULL e free nao e chamado — apenas palloc_free_page e executado, o que e correto e seguro.

Atencao: A busca em frame_free e $O(n)$ — percorre toda a lista. Para a fase atual isso e aceitavel. Quando evicao for implementada e frame_free for chamado com frequencia alta, uma hash table indexada por kpage eliminara esse custo.

5. Makefile.build — Ativacao da Compilacao VM

A ativacao da compilacao foi feita em duas etapas deliberadas: primeiro os arquivos foram criados com a linha comentada, depois a linha foi descomentada em commit separado. Isso separa **criar os arquivos** de **inclui-los no build** — útil para revisar a implementacao antes de integra-la ao kernel.

```
# Antes – nenhum codigo VM compilava:
#vm_SRC = vm/file.c
# Apos criar frame.c (ainda comentado):
# Virtual memory code.
#vm_SRC = vm/frame.c
# Apos descomentacao – frame.c entra no build:
# Virtual memory code.
vm_SRC = vm/frame.c
```

A linha **vm_SRC = vm/frame.c** instrui o sistema de build do PintOS a compilar e linkeditar frame.c junto com os demais módulos do kernel. Sem ela, mesmo que frame.h e frame.c existam, o kernel não teria acesso as funções frame_init, frame_alloc e frame_free.

6. Decisões de Projeto

Decisao	Alternativa Descartada	Justificativa
Lista encadeada simples para frame_table	Hash table ou array indexado por numero de frame	Suficiente para a fase atual sem evicao. Hash table sera util quando buscas frequentes por kpage forem necessarias.
Lock global unico (frame_lock)	Lock por frame ou estrutura lock-free	Menor complexidade. Gargalo aceitavel para cargas normais sem evicao concorrente intensa.
PAL_USER somado automaticamente em frame_alloc	Caller passa as flags completas	Evita erro de programacao. Todo frame na tabela e de usuario por definicao — a flag nao pode ser omitida.
palloc_free_page chamado mesmo sem entrada na tabela	Panic se frame nao encontrado	Compatibilidade com pagedir_destroy, que libera frames diretamente sem passar por frame_free.
owner = thread_current() no momento da alocao	Passar owner como parametro explicito	Simplifica a interface. O alocador e sempre a thread corrente — passar como parametro seria redundante.
malloc para frame_entry	Alocar com palloc ou na stack	A entrada precisa sobreviver ao retorno de frame_alloc. Stack e invalida apos retorno; palloc desperdicaria uma pagina inteira para uma struct pequena.

7. Proximos Passos — Projeto 3 Completo

A frame table e a fundacao. As funcionalidades seguintes do Projeto 3 dependem diretamente dela:

Componente	Arquivos	Dependencia da Frame Table
Tabela de Paginas Suplementar (SPT)	vm/page.h, vm/page.c	Chama <code>frame_alloc</code> ao resolver um page fault para obter um frame fisico e mapear a pagina virtual.
Evicao — Clock Algorithm	vm/frame.c (extensao)	<code>frame_alloc</code> deve chamar <code>frame_evict</code> quando <code>palloc</code> retorna NULL, percorrendo <code>frame_table</code> para escolher uma vitima.
Swap	vm/swap.h, vm/swap.c	Destino das paginas evictadas. <code>frame_free</code> e chamado apos a pagina ser escrita no swap.
Inicializacao no boot	threads/init.c	<code>frame_init()</code> deve ser registrado sob <code>#ifdef VM</code> apos <code>palloc_init()</code> e antes do primeiro processo de usuario.
Memory-mapped files	userprog/syscall.c	<code>SYS_MMAP</code> aloca frames via <code>frame_alloc</code> e os registra com <code>upage</code> correspondente ao offset no arquivo.

Fluxo previsto de um page fault com SPT: page fault handler consulta a SPT para o upage faltante → chama `frame_alloc(PAL_USER, upage)` → obtém `kpage` → carrega dado (do arquivo ou swap) em `kpage` → chama `pagedir_set_page(pagedir, upage, kpage, writable)` → retorna ao processo de usuario que causou o fault.